Fundamentals of Artificial Intelligence - Lecture 5: Constraint Satisfaction Problems (19 Oct)

Lecture overview

Lecture 5 introduces Constraint Satisfaction Problems (CSPs) as a structured alternative to generic search, where states are explicitly defined by variables with specific domains rather than treated as "black boxes." Unlike planning problems where action sequences and costs matter, CSPs focus on identification problems where only the final variable assignments satisfying constraints are relevant. The lecture illustrates this through examples like map coloring, N-Queens, cryptarithmetic, and Sudoku, highlighting how explicit constraint definitions enable powerful algorithms such as forward-checking to detect inconsistencies early. A key structural concept is the binary constraint graph, which visualizes variables as nodes and constraints as arcs, allowing solvers to exploit problem structure by identifying independent subproblems.

To optimize search efficiency, the lecture details several heuristics and consistency techniques. Strong k-consistency ensures that partial assignments can be extended without backtracking, though modern solvers often combine efficient arc consistency propagation with smart variable ordering strategies. The Minimum Remaining Values (MRV) heuristic employs a "fail-fast" approach by selecting the most constrained variable, while the Least Constraining Value (LCV) heuristic acts optimistically to preserve future flexibility for remaining variables. Additionally, value symmetry breaking is introduced to reduce the search space by eliminating redundant equivalent solutions through enforced ordering constraints. The session concludes by extending CSPs to Constrained Optimization Problems (COPs), which include an objective function, and introduces Branch-and-Bound as a meta-algorithm that maintains a dynamic bound on the best solution found to prune the search tree during backtracking.

Slide 1: Fundamentals of Artificial Intelligence

This lecture introduces the fundamentals of artificial intelligence, specifically focusing on constraint satisfaction problems. Presented by Professor Tias Guns as Lecture 05 for the 2026-27 academic year, the session utilizes foundational material adapted from the CS188 course at Berkeley.

Slide 2: FAI: Reasoning & decisions

The "FAI: Reasoning & decisions" course schedule outlines ten lectures covering algorithmic search fundamentals, constraint satisfaction, game trees, Markov decision processes, and logic-based planning. Key milestones include two project hackerspaces for practical application, a guest lecture on AI ethics, and a final exam information session. The curriculum progresses from deterministic search applications to stochastic environments, culminating in an analysis of broader AI implications.

Slide 3: RECAP: Designing Rational Agents

An agent is defined as an entity that perceives its environment through sensors and acts via actuators, while a rational agent specifically selects actions to maximize expected utility. The techniques used to determine these rational actions depend on the characteristics of the percepts, environment, and action space. Consequently, this course focuses on general AI search techniques for various problem types, emphasizing the ability to recognize when existing methods can solve new problems.

Slide 4: What is Search For?

Search operates under assumptions of a single agent with deterministic actions in a fully observed, discrete state space. It distinguishes between planning, where the specific sequence of actions and their varying costs or depths are critical, and identification problems, where only the final variable assignments matter. While heuristics provide problem-specific guidance for finding paths, some formulations treat all paths at the same depth as equivalent regardless of cost.

Slide 5: CSP Examples

The linked notebook demonstrates map coloring as a CSP: regions are variables, colors are domains, and neighboring regions must receive different colors. The URL is supplemental material; the model is the important idea.

Slide 6: Example: Map Coloring

The map coloring problem defines variables as regions, domains as available colors, and constraints requiring adjacent regions to have different colors. While explicit representations checking validity only upon full assignment are inefficient, a proactive approach immediately removes invalid colors from neighboring domains once a region is assigned. This forward-checking strategy detects inconsistencies earlier than standard backtracking, preventing the search from exploring paths that will inevitably fail due to shared constraints between unassigned variables.

Slide 7: Constraint Satisfaction Problems

While generic search problems treat states as arbitrary "black boxes" with flexible goal and successor functions, Constraint Satisfaction Problems (CSPs) represent a specialized subset where states are explicitly defined by variables X_i taking values from domains D. In CSPs, the goal test consists of constraints specifying allowable value combinations for variable subsets, a structured representation that enables general-purpose algorithms more powerful than standard search methods.

Slide 8: Example: N-Queens

Slide 8 introduces the N-Queens problem as a concrete example of constraint satisfaction formulation. It outlines "Formulation 1" by identifying the necessary components: the variables representing queen positions, their respective domains, and the constraints required to ensure no two queens attack each other. The slide lists these structural elements but does not provide specific definitions or detailed examples for them in this section.

Slide 9: Example: N-Queens

For the N-Queens formulation, choose one variable per column and let its domain be the possible rows. Constraints require queens to occupy distinct rows and avoid shared diagonals; the one-per-column representation already rules out column conflicts. This makes explicit which restrictions come from the variable representation and which must be checked as constraints.

Slide 10: Constraint Graphs

A constraint graph has one node per variable and an edge between variables whose values are restricted by a shared binary constraint. Dense or tightly connected regions indicate variables that interact strongly; disconnected components can sometimes be solved independently.

Slide 11: Constraint Graphs

A binary constraint satisfaction problem involves constraints relating at most two variables, which can be visualized as a binary constraint graph where nodes represent variables and arcs denote constraints. This graphical representation allows general-purpose algorithms to exploit the structure for faster search by identifying independent subproblems, such as treating Tasmania as a separate component in a map-coloring scenario.

Slide 12: Example: Cryptarithmetic

Cryptarithmetic assigns digits to letters so that a written arithmetic equation holds. Variables are letters, domains are digits, and constraints enforce column arithmetic and distinct digits where required.

Slide 13: Example: Sudoku

In the Sudoku example, variables correspond to each open square with a domain of integers from 1 to 9. Constraints are defined as 9-way `alldiff` requirements ensuring all values in every row, column, and block are distinct. Alternatively, these global constraints can be decomposed into numerous pairwise inequality constraints between individual squares.

Slide 14: Varieties of CSPs and Constraints

This section introduces CSP variations, including how variables, domains, and constraints can be represented. The chosen representation affects which propagation and search techniques are available.

Slide 15: Varieties of CSPs

Constraint Satisfaction Problems (CSPs) vary by variable type and domain size. Discrete CSPs with finite domains involve $O(d^n)$ complete assignments, exemplified by Boolean satisfiability, which is NP-complete. Infinite domains include integers or strings used in job scheduling, where linear constraints are solvable but nonlinear ones are undecidable. Continuous variables also appear in applications such as scheduling Hubble Telescope observations.

Slide 16: Varieties of Constraints

Constraints are categorized by the number of variables they involve: unary constraints affect a single variable (equivalent to domain reduction), binary constraints link pairs of variables, and higher-order constraints involve three or more variables, such as alldiff or scheduling requirements. Additionally, soft constraints represent preferences where one assignment is better than another, often modeled as costs that transform the problem into a constrained optimization task.

Slide 17: Real-World CSPs

Real-world Constraint Satisfaction Problems (CSPs) encompass diverse applications including assignment problems like allocating instructors to classes, and timetabling challenges determining when and where courses are offered. Additional critical domains involve hardware configuration, transportation and factory scheduling, circuit layout design, and fault diagnosis, with further demonstrations planned for the end of the class.

Slide 18: Solving CSPs

Solving a CSP means finding a complete assignment that satisfies every constraint. The following search slides compare uninformed exploration with backtracking and constraint-based pruning.

Slide 19: Standard Search Formulation

The standard search formulation for Constraint Satisfaction Problems (CSPs) defines states as partial assignments of values to variables. The process begins with an empty initial state, uses a successor function that assigns a value to any unassigned variable, and applies a goal test requiring the assignment to be both complete and constraint-satisfying. This straightforward, naïve approach serves as the baseline before introducing subsequent improvements.

Slide 20: Search Methods

Breadth-first search would expand partial assignments in order of depth, using a queue. The map-coloring branches illustrate how many partial assignments can be generated before a complete consistent assignment is reached; BFS uses substantial memory, so CSP solvers usually favor backtracking with propagation and variable-ordering heuristics.

Slide 21: Search Methods

The prompts ask what breadth-first and depth-first search would do on partial CSP assignments. The next pages use the map-coloring tree to show why search order and early consistency checks matter.

Slide 22: Demo

This section presents a demonstration of Constraint Satisfaction Problem (CSP) backtracking, accessible via the provided CMU course link. The demo visually illustrates how the backtracking algorithm explores variable assignments and handles constraint violations during the search process. Students are directed to this external resource to observe the dynamic execution of the algorithm in action.

Slide 23: Backtracking Search

Backtracking search assigns one variable at a time using depth-first search. After each assignment, it checks consistency with earlier choices and abandons the branch as soon as a constraint is violated, avoiding work on assignments that cannot lead to a solution.

Slide 24: Backtracking Search

Backtracking search improves depth-first search by considering only one variable per node, fixing an ordering to eliminate redundant permutations since assignments are commutative. It employs an incremental goal test that checks constraints immediately, ignoring inconsistent values before expanding branches rather than waiting for a full assignment. This approach requires some computation but significantly reduces the branching factor and avoids exploring invalid states early in the search tree.

Slide 25: Backtracking Example

This title introduces a worked backtracking example. Backtracking assigns a variable, checks consistency, and returns to the previous choice when the partial assignment cannot be completed.

Slide 26: Demo

This is a link to the course’s interactive CSP backtracking demonstration. Use it as a worked visualization of assignments and constraint checks; the URL itself is supplementary material, not a separate algorithm.

Slide 27: Backtracking Search

Backtracking search is defined as a combination of Depth-First Search (DFS), value-consistency checks, and an immediate failure mechanism upon constraint violation. This formulation highlights that while the core traversal and consistency logic are established, significant algorithm design choices remain regarding how these components are specifically implemented or ordered.

Slide 28: Improving Backtracking

General-purpose improvements to backtracking yield significant speed gains through strategic ordering and filtering. Ordering decisions involve selecting the next variable to assign and determining the sequence for trying its values, such as prioritizing variables with the most impact or choosing values that rule out the fewest options in other variables. Additionally, filtering techniques aim to detect inevitable failure early by targeting bottlenecks first, a "fail-fast" approach that often eliminates the need for further search once difficult constraints are resolved.

Slide 29: Filtering

Filtering is the process of maintaining the domains of unassigned variables by systematically eliminating invalid values. This technique involves identifying and crossing off "bad options" that violate constraints, thereby narrowing the search space to only feasible assignments for the remaining variables in the problem.

Slide 30: Filtering: Forward Checking

Forward checking is a filtering technique designed to avoid dead ends by explicitly tracking the domains of unassigned variables. As each variable is assigned a value, this method immediately crosses off any remaining options in other variables that would violate constraints with the current assignment. By eagerly removing these impossible combinations, forward checking detects inconsistencies early before they propagate further into the search process.

Slide 31: Demo

The same backtracking demonstration link reappears during the worked example. It points to the interactive assignment search and does not introduce a new CSP concept.

Slide 32: Filtering: Constraint Propagation

While forward checking propagates information from assigned to unassigned variables, it fails to detect all inconsistencies early, such as when NT and SA cannot both be blue. Constraint propagation addresses this limitation by reasoning about the consistency of every constraint rather than just immediate neighbors. By removing invalid values like blue or red from the domains of neighboring decision variables immediately upon assignment, this method ensures the early detection of all failures that forward checking might miss.

Slide 33: Consistency of A Single Arc

An arc $X \to Y$ is consistent if every value in the tail (X) has at least one compatible value in the head (Y) that satisfies the constraint; otherwise, the inconsistent values are deleted from the tail's domain. While forward checking enforces this property only for arcs pointing to a new assignment, full arc consistency checks all arcs in both directions. For example, if assigning Western Australia red leaves no valid color for Northern Territories, that specific value is removed from Northern Territories' domain to maintain consistency.

Slide 34: Arc Consistency of an Entire CSP

Arc consistency for an entire CSP involves propagating constraints to ensure all arcs are consistent, requiring that neighbors of any variable losing a value be rechecked. This method detects failures earlier than forward checking and can function as a preprocessor or after each assignment. The process specifically requires deleting values from the tail of the domain list.

Slide 35: Enforcing Arc Consistency in a CSP

Enforcing arc consistency in a Constraint Satisfaction Problem (CSP) typically requires a runtime of $O(n^2d^3)$, where n represents the number of variables and d the domain size. However, this computational complexity can be optimized to $O(n^2d^2)$ using more efficient algorithms. This reduction in time complexity is significant for improving the performance of constraint propagation in larger problem instances.

Slide 36: Demo

This repeated demo reference continues the CSP walkthrough. It is a resource pointer; the lecture's algorithmic point remains incremental assignment with consistency checks and pruning.

Slide 37: Limitations of Arc Consistency

Enforcing arc consistency does not guarantee a unique solution, as it may leave one, multiple, or even no solutions while failing to detect the latter. Because arc consistency only verifies pairwise constraints on edges rather than global satisfiability, it cannot identify inconsistencies that require deeper search. Consequently, arc consistency must be integrated within a backtracking search algorithm to fully resolve these limitations and find valid assignments.

Slide 38: K-Consistency

K-consistency defines increasing degrees of constraint satisfaction where 1-consistency (node consistency) ensures a single node's domain satisfies unary constraints, and 2-consistency (arc consistency) guarantees any assignment to one node in a pair can extend to the other. Generalizing this, k-consistency requires that any consistent assignment to $k-1$ nodes can be extended to a kth node. While higher values of k provide stronger consistency guarantees, they become progressively more expensive to compute.

Slide 39: Strong K-Consistency

Strong k-consistency implies that a problem is also (k-1), (k-2), down to 1-consistent, with strong n-consistency guaranteeing a solution without backtracking by ensuring any partial assignment can be extended. While higher-order consistency like path consistency (k=3) offers a middle ground between arc consistency and full n-consistency, modern solvers typically combine efficient arc consistency propagation with smart search strategies that intelligently select the next variable to check.

Slide 40: Ordering

Variable ordering decides which unassigned variable to select next; value ordering decides which domain value to try first. Heuristics such as MRV and least-constraining value use these choices to reduce search.

Slide 41: Ordering: Minimum Remaining Values

The Minimum Remaining Values (MRV) heuristic, also known as the "most constrained variable" strategy, selects the variable with the fewest legal values remaining in its domain. This approach implements a "fail-fast" ordering by prioritizing variables with the least flexibility, which helps the algorithm quickly identify inconsistencies or find a solution rather than exploring paths with many available options.

Slide 42: Ordering: Least Constraining Value

The Least Constraining Value heuristic selects the value for a chosen variable that rules out the fewest options for remaining variables, acting as an optimistic strategy rather than a "fail-fast" approach. Although determining this value requires additional computation, such as rerunning filtering, it maximizes future flexibility by preserving more possibilities. This value ordering can be effectively combined with different variable ordering strategies to optimize the search process.

Slide 43: Demo

The final repeated demo reference returns to the same backtracking visualization. The URL is retained as a study aid, with no additional theoretical claim on this page.

Slide 44: Value Symmetry

Value symmetry in Constraint Satisfaction Problems occurs when permuting values, such as swapping colors in map-coloring, generates multiple equivalent solutions without changing validity. To reduce the search space, symmetry breaking constraints are added by assigning numeric values to domains and enforcing an ordering, such as requiring Western Australia's color value to be lower than Northern Territory's. This optimization is critical because it prevents the algorithm from redundantly exploring symmetric infeasible branches of the search tree where no solution exists, not just avoiding duplicate valid solutions.

Slide 45: Constraint Optimisation?

Most real-world problems are framed as optimization challenges involving variables with finite domains, specific constraints, and an objective function. These elements collectively define a Constrained Optimization Problem (COP), which serves as the formal structure for addressing such complex scenarios where solutions must satisfy restrictions while optimizing a target value.

Slide 46: Modeling

Modeling a problem involves defining variables with specific domains, establishing constraints over those variables, and optionally specifying an objective function to optimize. The knapsack example illustrates this by assigning binary domains to item variables, constraining their total weight to 15, and maximizing total value. Once these components are defined, the model is executed using a solve method to find the optimal solution.

Slide 47: Optimisation: branch-and-bound

Branch-and-bound solves an optimization problem by tracking the best complete solution found and pruning any partial branch whose bound cannot improve it. For minimization, a lower bound at least as large as the current best cost makes the branch uncompetitive.

Slide 48: https://cpmpy.readthedocs.io/

The CPMPy documentation link points to a constraint-programming library. It supports expressing variables, domains, constraints, and objectives declaratively; the documentation is a practical reference rather than a new solving method.

Slide 49: Modeling

Modeling a problem like the Knapsack example involves defining binary variables (e.g., gr, bl), establishing constraints on their combinations (such as a weight limit of 15), and optionally specifying an objective function to maximize. A complete model consists of these variables with defined domains, the governing constraints, and the optional optimization goal, which is then executed using a solve method.

Slide 50: Integrated constraint solving

Integrated constraint solving highlights the numerous ways constraint solving techniques are combined with machine learning. A specific illustration of this synergy is provided by Visual Sudoku, an interactive demonstration available at visualsudoku.cs.kuleuven.be that showcases how these methods work together in practice.

Slide 51: CPMpy demo

CPMpy is a modern Constraint Satisfaction and Optimization library available at cpmpy.readthedocs.io. The slide highlights key demonstrations, including a graph coloring example for Australia, an N-queens problem solver capable of handling up to 1000 queens, and various industrial examples featuring visualizations on toy data. All these practical implementations are accessible via the provided GitHub repository links within the CPMpy examples directory.

Slide 52: Summary

Constraint Satisfaction Problems (CSPs) are a specialized form of search problem where states represent partial assignments and the goal test is defined by constraints. The fundamental solution approach is backtracking search, which can be significantly accelerated through filtering, variable/value ordering, exploiting problem structure, and symmetry breaking. These techniques enable CSPs to address numerous practical applications effectively.

Key points to remember

To optimize search efficiency, the lecture details several heuristics and consistency techniques. Strong k-consistency ensures that partial assignments can be extended without backtracking, though modern solvers often combine efficient arc consistency propagation with smart variable ordering strategies. The Minimum Remaining Values (MRV) heuristic employs a "fail-fast" approach by selecting the most constrained variable, while the Least Constraining Value (LCV) heuristic acts optimistically to preserve future flexibility for remaining variables. Additionally, value symmetry breaking is introduced to reduce the search space by eliminating redundant equivalent solutions through enforced ordering constraints. The session concludes by extending CSPs to Constrained Optimization Problems (COPs), which include an objective function, and introduces Branch-and-Bound as a meta-algorithm that maintains a dynamic bound on the best solution found to prune the search tree during backtracking.